Lecture 23
Deductive and Transformational Synthesis

Armando Solar-Lezama
with some slides from Adam Chlipala

A Transformation System for Developing Recursive Programs

R.M. Burstall and John Darlington 1977

“We make no claim for any sort of completeness of the system; it embodies only one family of program transformations, and we have no formal delineation of this family. … However, we hope that the examples will give the reader pleasure…”

Key premise

(1) Start with a clean high-level representation of the problem

(2) Apply semantics preserving transformations

(3) Arrive at a less clean but more efficient representation

Clean high-level spec

$$dot\big(x, y, 0\big) \Leftarrow 0$$
$$dot\big(x, y, n+1\big) \Leftarrow dot\big(x, y, n\big) + x\big[n+1\big]y[n+1]$$

$$fib(0) \Leftarrow 0 fib(1) \Leftarrow 1$$
$$fib(x+2) \Leftarrow fib(x+1) + fib(x)$$

$$concat(nil, z) \Leftarrow zconcat\Big(cons\big(x, y\big), z\Big) \Leftarrow cons(x, concat\big(y, z\big))$$

Example

$$dot\big(x, y, 0\big) \Leftarrow 0$$
$$dot\big(x, y, n+1\big) \Leftarrow dot\big(x, y, n\big) + x\big[n+1\big]\, y[n+1]$$

$$f(a, b, c, d, n) \Leftarrow dot(a, b, n) + dot(c, d, n)$$

Key transformation rules

Instantiation

- $f(a,b,c,d,n) \Leftarrow dot(a,b,n) + dot(c,d,n)$
- $f(a,b,c,d,0) \Leftarrow dot(a,b,0) + dot(c,d,0)$
- $f(a,b,c,d,n+1) \Leftarrow dot(a,b,n+1) + dot(c,d,n+1)$

Unfolding

- $f(a,b,c,d,n+1) \Leftarrow dot(a,b,n+1) + dot(c,d,n+1)$
- $f(a,b,c,d,n+1) \Leftarrow dot(a,b,n) + a[n+1]b[n+1] + dot(c,d,n) + c[n+1]d[n+1]$

Folding

- $f(a,b,c,d,n+1) \Leftarrow dot(a,b,n) + a[n+1]b[n+1] + dot(c,d,n) + c[n+1]d[n+1]$
- $f(a,b,c,d,n+1) \Leftarrow f(a,b,c,d,n) + a[n+1]b[n+1] + c[n+1]d[n+1]$

Key transformation rules

Algebraic laws

- $f(a,b,c,d,n+1) \Leftarrow dot(a,b,n) + a\left[n+1\right]b\left[n+1\right] + dot(c,d,n) + c\left[n+1\right]d[n+1]$
- $f(a,b,c,d,n+1) \Leftarrow dot(a,b,n) + dot(c,d,n) + a\left[n+1\right]b\left[n+1\right] + c\left[n+1\right]d[n+1]$

Abstraction

- $E\left[F_1 \to u_1, \ldots F_n \to u_n\right]\ where\ \left\langle u_1 \ldots u_n \right\rangle = \left\langle F_1, \ldots, F_n \right\rangle$

Fibonacci

1. $f(0)$	$\Leftarrow 1$	given
2. $f(1)$	$\Leftarrow 1$	given
3. $f(x+2)$	$\Leftarrow f(x+1) + f(x)$	given
4. $g(x)$	$\Leftarrow \langle f(x+1), f(x)\rangle$	definition (eureka)
5. $g(0)$	$\Leftarrow \langle f(1), f(0)\rangle$	instantiation
	$\Leftarrow \langle 1, 1\rangle$	unfolding with 1 and 2
6. $g(x+1)$	$\Leftarrow \langle f(x+2), f(x+1)\rangle$	instantiate 4
	$\Leftarrow \langle f(x+1) + f(x), f(x+1)\rangle$	unfold with 3
	$\Leftarrow \langle u+v, u\rangle$ **where** $\langle u, v\rangle = \langle f(x+1), f(x)\rangle$	abstract
	$\Leftarrow \langle u+v, u\rangle$ **where** $\langle u, v\rangle = g(x)$	fold with 4
7. $f(x+2)$	$\Leftarrow u+v$ **where** $\langle u, v\rangle = \langle f(x+1), f(x)\rangle$	abstract 3
	$\Leftarrow u+v$ **where** $\langle u, v\rangle = g(x)$	fold with 4

Dreams => Programs

Waldinger & Manna 1979

Deductive synthesis

Similar ideas to Burstall and Darlington with a solid theoretical footing

A lot more ambitious

> *"…Such a system accepts specifications that express the purpose of the program to be constructed, without giving any hint of the algorithm to be employed. With no further human intervention, the system attempts to transform these specifications into a program that achieves the expressed purpose"*

Specifications

lessall(x l) $\Longleftarrow$ **compute** $x < all(l)$
where x is a number and
l is a list of numbers.

gcd(x y) $\Longleftarrow$ **compute** $max\{z : z|x \text{ and } z|y\}$
where x and y are nonnegative integers and
$x \neq 0$ or $y \neq 0$.

Basic transformation rules

$t \Rightarrow t'$

$t \Rightarrow t' \quad \text{if } P$

$\mathit{true}\ \mathit{and}\ Q \Rightarrow Q$

$u|v \Rightarrow \mathit{true} \quad \text{if } u \text{ is an integer and } v = 0$

Derivation

Goal 1: **compute** $x <$ *all(l)*.

$$lessall(x\ l) \Longleftarrow \textbf{compute}\ x < all(l)$$
where x is a number and
l is a list of numbers.

Rule 1: $P(all(l)) \Rightarrow true$ if l is the empty list

Goal 2: **prove** l is the empty list. ✖

Rule 2: Conditional formation

$S\ where\ Q \Rightarrow if(P)\ S\ where\ P\ and\ Q$

$lesall(x,l) \Leftarrow$ *if empty(l) compute* $x < all(l)$ *where l is empty*
else compute $x < all(l)$ *where l is not empty*

Derivation

Rule 1: $P(all(l)) \Rightarrow true$ if l is the empty list

$$lesall(x, l) \Leftarrow if\ empty(l)\ \ compute\ x < all(l)\ where\ l\ is\ empty$$
$$else\ compute\ x < all(l)\ where\ l\ is\ not\ empty$$

Rule 3: $P(all(l)) \Rightarrow P(head(l))\ and\ P(all(tail(l)))$
if l is a nonempty list.

$$lesall(x, l) \Leftarrow if\ empty(l)\ \ true \qquad else\ compute\ x < a$$

Goal 4: **compute** $x < head(l)\ and\ x < all(tail(l))$.

$$lesall(x, l) \Leftarrow if\ empty(l)\ \ true \qquad else\ x < head(l)\ and$$

Rule 4: Given $F(x) \Rightarrow compute\ P(x) when\ Q$
$P(t) \Rightarrow F(t)\ if\ Q(t) and\ F(t) terminates$

Challenges

Proving that conditions are satisfied could be non-trivial

Transformation rules can be very domain-specific and you need a lot of them

Knowing what to apply when can be challenging

- Very large space of possible derivations

Modern incarnations

Fiat: Deductive Synthesis of Abstract Data Types in a Proof Assistant

- Benjamin Delaware, Clément Pit--Claudel, Jason Gross, Adam Chlipala

Rely on modern proof assistant to help automate routine parts of the derivation

Lift the level of abstraction of transformation rules so they are easier to apply

Fiat: A Program Derivation Framework in Coq

Program synthesis by stepwise refinement

1. Write mathematical specifications without worrying about performance.
2. Apply **optimization scripts** to refine specifications into efficient programs.
3. Coq's usual proof checker validates that scripts have preserved semantics.

Big delta from past related work:
“high automation without giving up high assurance”



Step 1: Specification

```
query "NumOrders" (author : string) : nat :=
  Count (For (o in “Orders”) (b in “Books”)
        Where (author = b!”Author”)
        Where (b!”ISBN” = o!”ISBN”)
        Return ())
```

Notes:

- Example of an SQL-style query
- Similar to monadic query comprehension notation.
- Parsed & type-checked within normal Coq code (extensible parser, etc.)

Step 2: Optimization Script

Choose data structures:

```
Definition BookStorage : IndexFor Book.
  mkIndex [ BookSchema/sAUTHOR; BookSchema/sISBN ].
Defined.
Definition OrderStorage : IndexFor Order.
  mkIndex [ OrderSchema/sISBN ].
Defined.
```

Define *abstraction relation* connecting them to specification:

```
Definition Bookstore_AbsR := …
```

Call library tactic to implement specs in terms of this relation:

```
plan Bookstore_AbsR.
```

Step 3: Reasonable Code!

```
meth “NumOrders” (p : rep, s : string) : nat :=
  ret (p, fold_left
    (fun (count : nat) (x : Tuple) =>
      count + bcount (snd p) (Some x!”ISBN”, []))
    (bfind (fst p) (Some s, (None, []))) 0)
```

Calls to methods of library data structures (nested AVL trees)

Modern Transformation systems

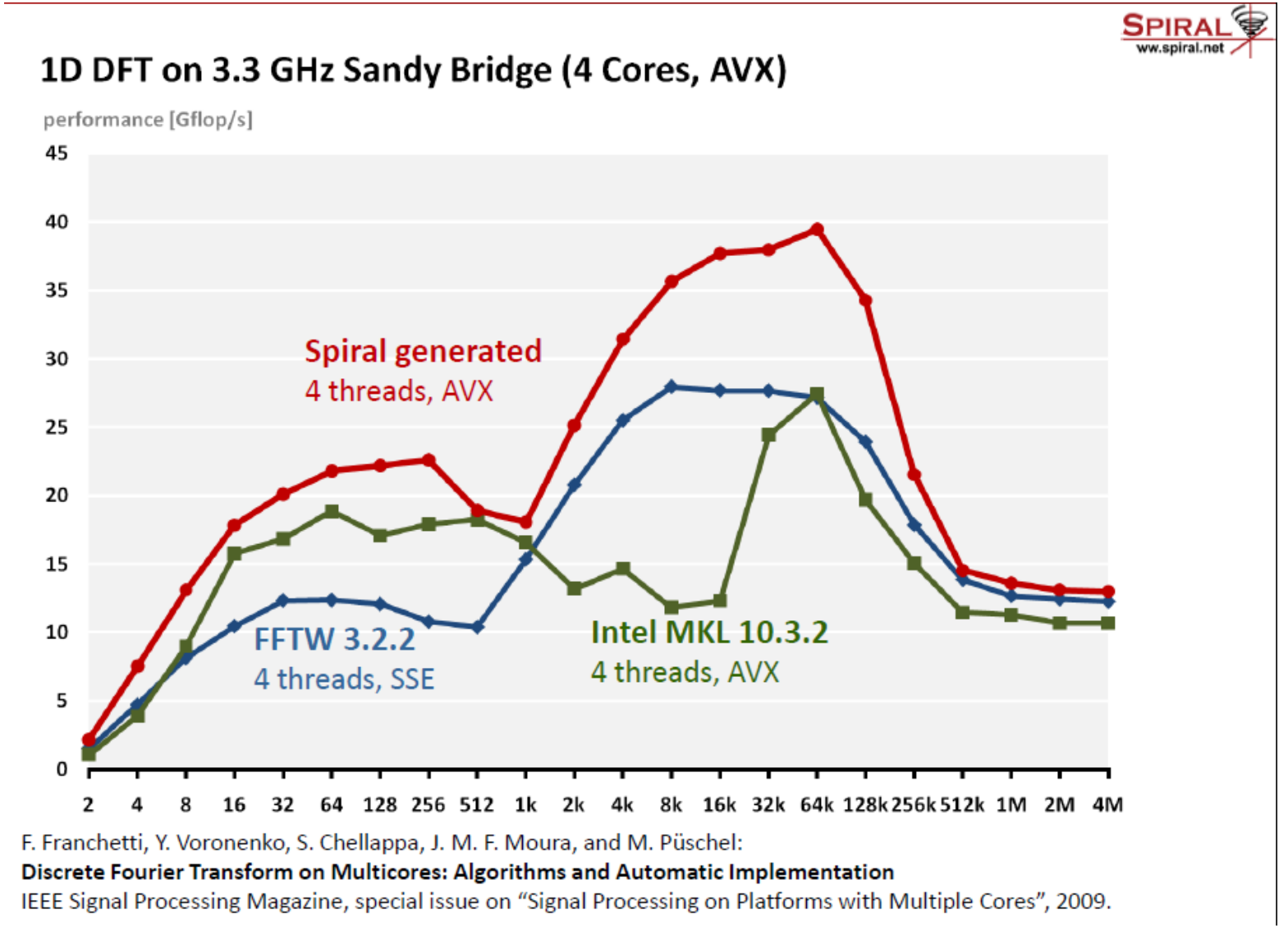
Spiral

- Led by Markus Puschel, Franz Franchetti and Jose Moura

Key ideas:

- Focus on a specific domain
- Build in heuristics and transformations that capture domain insights
- Throw a lot of computing power!

Spiral performance



F. Franchetti, Y. Voronenko, S. Chellappa, J. M. F. Moura, and M. Püschel:
Discrete Fourier Transform on Multicores: Algorithms and Automatic Implementation
IEEE Signal Processing Magazine, special issue on "Signal Processing on Platforms with Multiple Cores", 2009.